

Estudiante: Julio Alejandro Mazo Reyes.

Materia: Programación Paralela.

Parcial 1

Video: https://youtu.be/Ah-lD1_LopU

Repositorio: https://github.com/JulioM-108/Parcial_Paralela

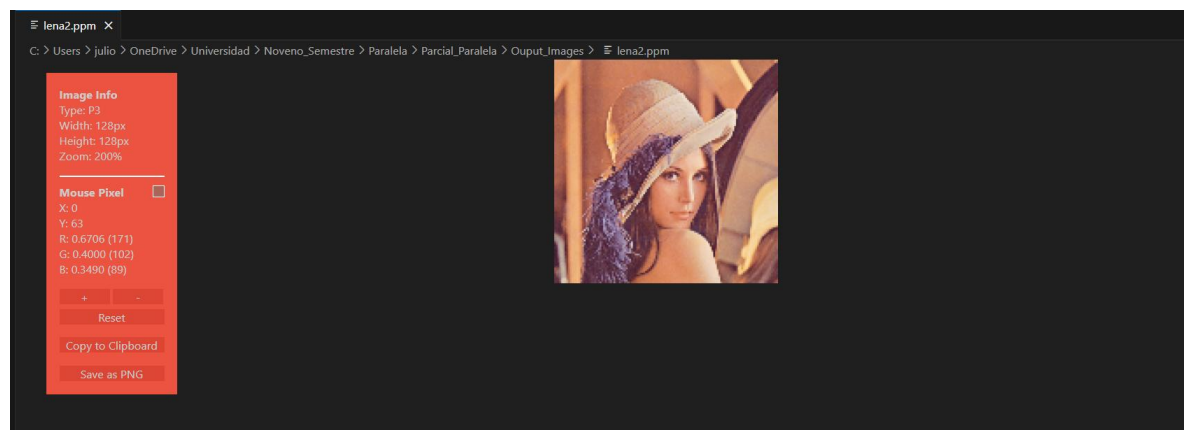
¿Cómo influye la programación paralela, utilizando OpenMP ó Pthreads y MPI (En un entorno simulado con docker), en el tiempo de ejecución del filtrado de imágenes PPM y PGM en comparación con una implementación secuencial?

R: En este punto me pareció que la versión secuencial funciono un poco parecido a la paralela. Por un lado, con Pthreads, los tiempos de CPU andan entre 0.04 y 0.32 segundos, lo cual ya es una mejora notable al dividir el trabajo entre hilos. Por otro lado, con OpenMP, los tiempos para la imagen sulfur.pgm fueron de 1.6138 segundos, su gran ventaja (que no fue el tiempo realmente) es que se me hizo más fácil de implementar para paralelizar bucles, lo que agilizo los procesos.

Pero el que realmente me impresionó fue MPI en ese entorno de Docker, con tiempos de 0.023 a 0.024 segundos por nodo. Esto demuestra que al distribuir el trabajo en varios procesos que actúan como si fueran nodos separados, se consigue la mayor reducción de tiempo. En conclusión, todos estos enfoques, Pthreads, OpenMP y MPI, demuestran que la programación paralela es la clave para optimizar el uso de la CPU y la memoria, logrando que el filtrado de imágenes sea mucho más rápido que si se hiciera de forma secuencial.

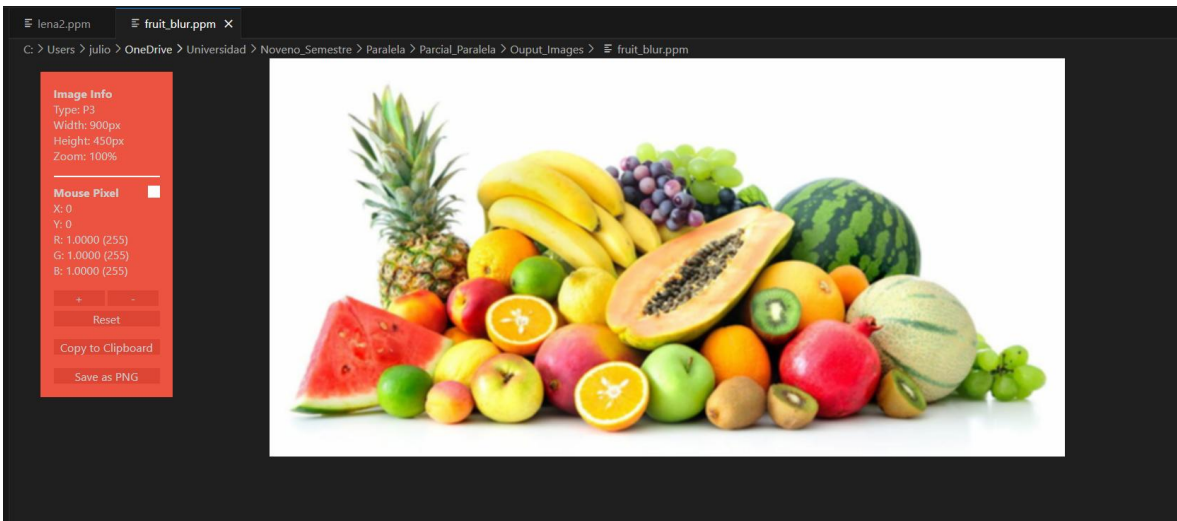
Versión Básica de Procesador:

```
japeto@fe8a305380a1:~$ ./proceso Images/lena.ppm Output_Images/lena2.ppm
Imagen copiada exitosamente.
japeto@fe8a305380a1:~$ |
```

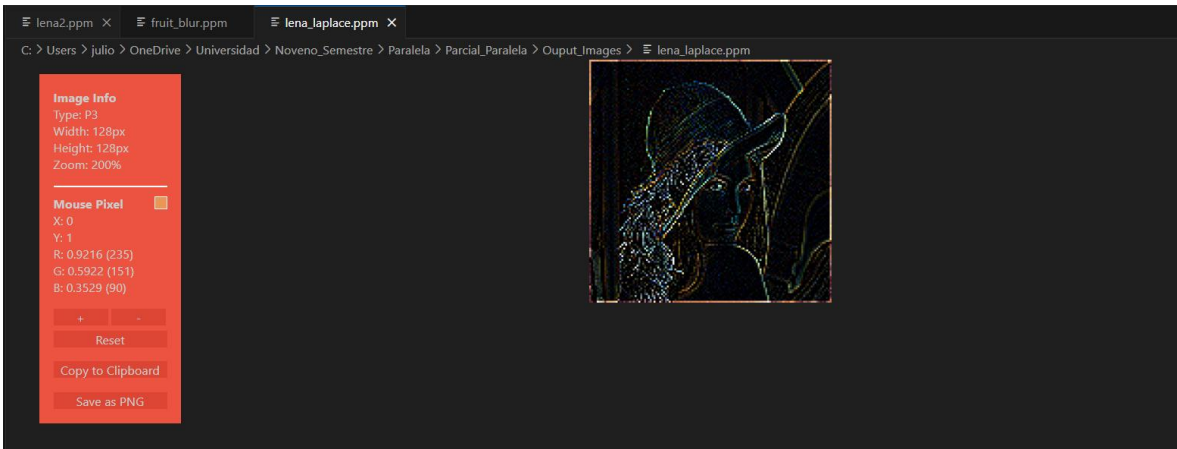


Versión Secuencial:

```
japeto@fe8a305380a1:~$ ./filtro Images/fruit.ppm Ouput_Images/fruit_blur.ppm --f blur
Tiempo de CPU usado en el filtrado: 0.075302 segundos
Imagen procesada con filtro blur y guardada en Ouput_Images/fruit_blur.ppm
japeto@fe8a305380a1:~$ |
```



```
japeto@fe8a305380a1:~$ ./filtro Images/lena.ppm Ouput_Images/lena_laplace.ppm --f laplace
Tiempo de CPU usado en el filtrado: 0.005597 segundos
Imagen procesada con filtro laplace y guardada en Ouput_Images/lena_laplace.ppm
japeto@fe8a305380a1:~$ |
```

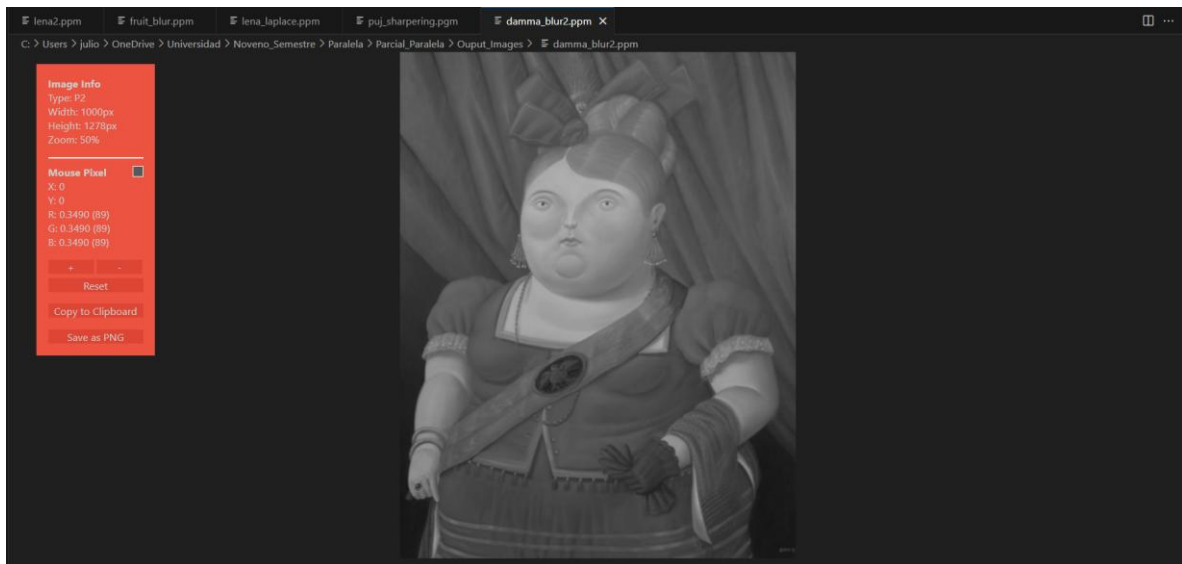


```
japeto@fe8a305380a1:~$ ./filtro Images/puj.pgm Ouput_Images/puj_sharpering.pgm --f sharpen
Tiempo de CPU usado en el filtrado: 0.075964 segundos
Imagen procesada con filtro sharpen y guardada en Ouput_Images/puj_sharpering.pgm
japeto@fe8a305380a1:~$ |
```

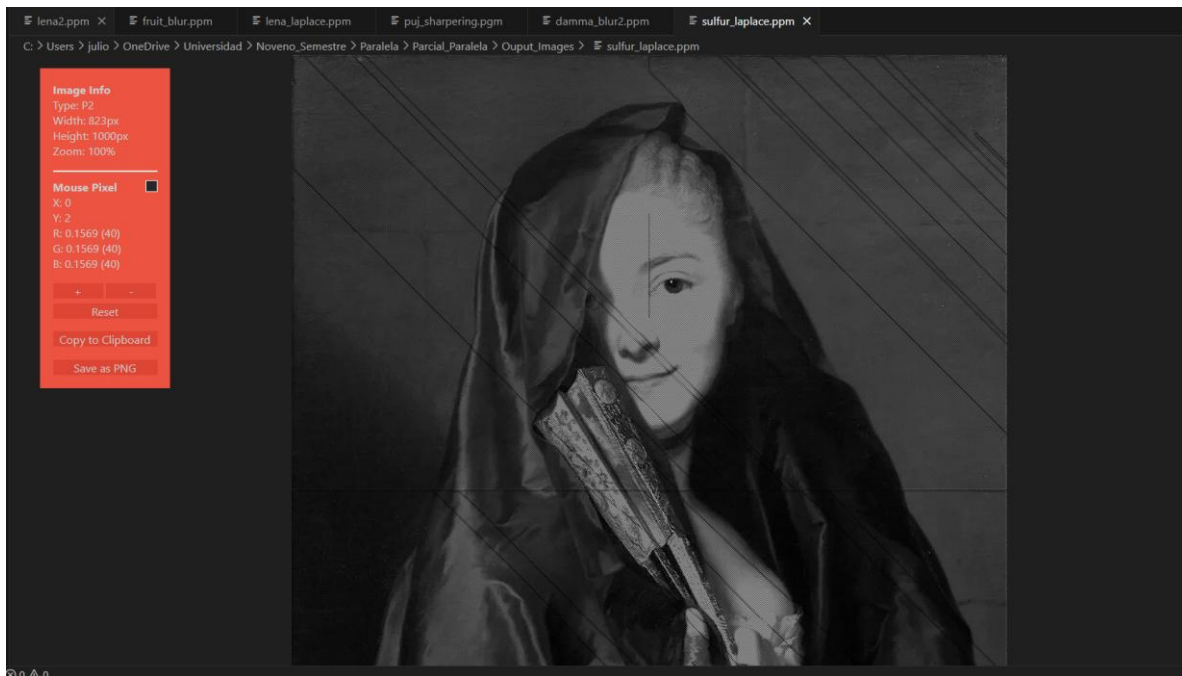


Phtreads:

```
japeto@fe8a305380a1:~$ ./Ejecutables/filtro_pth Images/damma.ppm Ouput_Images/damma_blur2.ppm --f blur
Tiempo de CPU con pthreads: 0.323105 segundos
Imagen procesada con filtro blur y guardada en Ouput_Images/damma_blur2.ppm
japeto@fe8a305380a1:~$
```



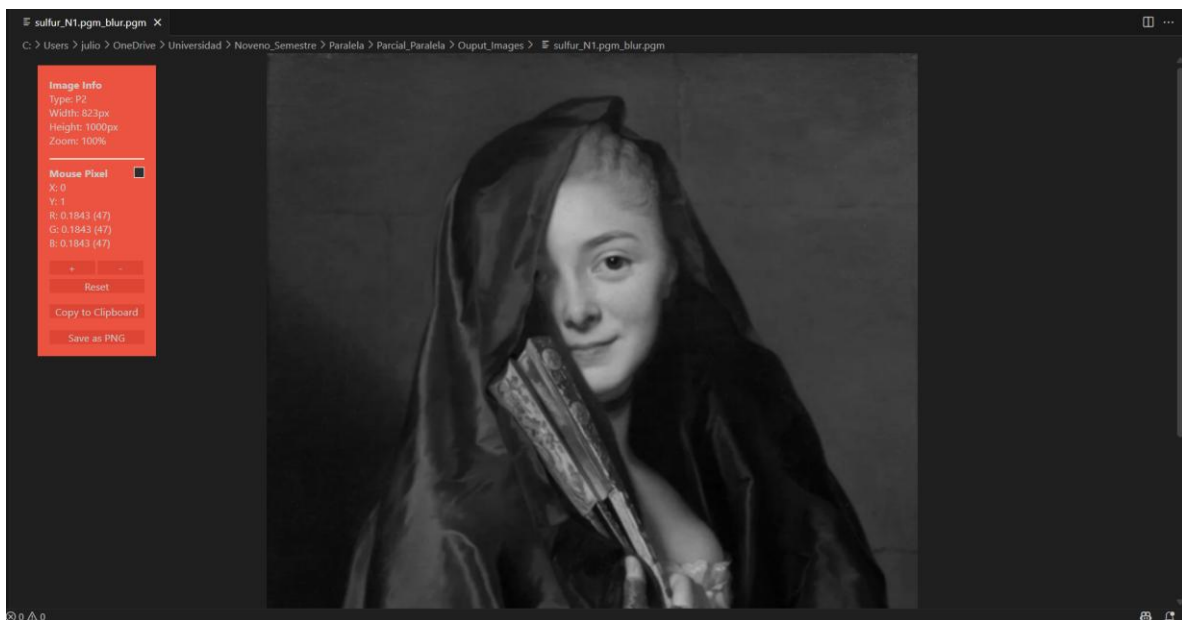
```
Tiempo de CPU con pthreads: 0.047752 segundos
Imagen procesada con filtro laplace y guardada en Ouput_Images/sulfur_laplace.ppm
```



OMP:

Nota: Se corrigió el formato y se arregló la salida de los archivos.

```
japeto@fe8a305380a1:~$ g++ -fopenmp -o filtro_omp src/filterer_omp.cpp
japeto@fe8a305380a1:~$ ./Ejecutables/filtro_omp Images/sulfur.pgm Ouput_Images/sulfur_N1.pgm
Filtro blur aplicado y guardado en Ouput_Images/sulfur_N1.pgm_blur.pgm
Filtro laplace aplicado y guardado en Ouput_Images/sulfur_N1.pgm_laplace.pgm
Filtro sharpen aplicado y guardado en Ouput_Images/sulfur_N1.pgm_sharpen.pgm
Tiempo total con OpenMP: 1.61308 segundos
japeto@fe8a305380a1:~$ |
```



MPI:

```
japeto@master:/tmp/build$ mpirun -np 4 --allow-run-as-root /tmp/build/filterer_mpi /app/imagenes/lena.ppm /app/output/lena_filtered.ppm --f blur
[master:00262] Read -1, expected 98304, errno = 38
[master:00259] Read -1, expected 98304, errno = 38
[master:00256] Read -1, expected 98304, errno = 38
Rank 0: wrote output /app/output/lena_filtered.ppm_rank0.ppm
=== Tiempos por nodo ===
Rank 0: CPU time = 0.023336 s, wall time = 0.0234736 s
Rank 1: CPU time = 0.023136 s, wall time = 0.0232487 s
Rank 2: CPU time = 0.023288 s, wall time = 0.0234098 s
Rank 3: CPU time = 0.023816 s, wall time = 0.0239472 s
japeto@master:/tmp/build$
```

